

Verteilte Systeme, Containerisierung

Vorwort:

Grundlage für diese Aufgabe ist die Präsentation «Kubernetes Einstieg»

Dauer: 100 Minuten

Besprechung: 20 Minuten

Total: 120 Minuten

In diesem Mini-Lab bauen Sie einen lokalen Kubernetes-Cluster mit k3d auf und deployen darin eine erste Standard-Applikation. Ziel ist nicht, alle Details auswendig zu lernen, sondern das Grundmodell von Cluster, Nodes, Pods, Deployments und Services praktisch zu sehen.

Offizielle Dokumentation

<https://k3d.io/>

<https://kubernetes.io/docs/concepts/>

<https://kubernetes.io/docs/reference/kubectl/>

<https://kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/>

Aufgabe 1)

Kubernetes Mental Model

Ordnen Sie die wichtigsten Begriffe, bevor Sie den Cluster starten.

a) Erklären Sie die Begriffe Cluster, Node, Pod, Deployment, Service und Namespace in je einem Satz.

Cluster: Ein Kubernetes-Cluster ist die Gesamtheit aller Ressourcen, die zusammen Container-Anwendungen ausführen.

Node: Ein Node ist ein einzelner Server (physisch oder virtuell), auf dem Pods ausgeführt werden.

Pod: Ein Pod ist die kleinste ausführbare Einheit in Kubernetes und enthält einen oder mehrere Container.

Deployment: Ein Deployment verwaltet Pods und stellt sicher, dass die gewünschte Anzahl von Pods jederzeit läuft.

Service: Ein Service stellt Pods über eine stabile Netzwerkadresse erreichbar zur Verfügung.

Namespace: Ein Namespace dient zur logischen Trennung und Organisation von Ressourcen innerhalb eines Clusters.

b) Was ist der wichtigste Unterschied zwischen Docker Compose und Kubernetes?

Docker Compose wird hauptsächlich verwendet, um mehrere Container auf einem einzelnen Host einfach zu starten und zu verwalten.

Kubernetes ist eine umfassende Orchestrierungsplattform für Container, die Container über mehrere Nodes hinweg verwalten, skalieren und automatisch wiederherstellen kann.

c) Erklären Sie kurz die Idee von gewünschtem Zustand, aktuellem Zustand und Controller.

- **Gewünschter Zustand (Desired State):** Beschreibt, wie die Anwendung laut Konfiguration laufen soll (z. B. 3 Pods).
- **Aktueller Zustand (Current State):** Beschreibt, wie die Anwendung momentan tatsächlich läuft (z. B. nur 2 Pods).
- **Controller:** Überwacht beide Zustände und sorgt dafür, dass der aktuelle Zustand wieder dem gewünschten Zustand entspricht, indem fehlende Ressourcen automatisch erstellt oder repariert werden.

Aufgabe 2)

k3d Cluster aufbauen

Erstellen Sie einen lokalen Kubernetes-Cluster mit einem Server und zwei Worker-Nodes.

a) Prüfen Sie, ob Docker, k3d und kubectl verfügbar sind. Notieren Sie die verwendeten Befehle.

Tipp: docker version / k3d version / kubectl version --client

k3d: <https://k3d.io/stable/#installation>

```
winget install -e --id Docker.DockerDesktop
```

```
docker version
```

```
k3d version
```

```
kubectl version --client
```

b) Erstellen Sie den Cluster «teko-k8s» mit zwei Agent-Nodes.

Tipp: k3d cluster create teko-k8s --agents 2

```
k3d cluster create teko-k8s --agents 2
```

c) Prüfen Sie Cluster, Docker-Container und Kubernetes-Nodes. Was sehen Sie?

Tipp: k3d cluster list / docker ps / kubectl get nodes -o wide

```
k3d cluster list
```

```
docker ps
```

```
kubectl get nodes -o wide
```

d) Welcher kubectl-Kontext ist aktiv und warum ist das wichtig?

Tipp: kubectl config current-context

```
kubectl config current-context
```

Aufgabe 3)

Namespace und Deployment

Arbeiten Sie in einem eigenen Namespace und starten Sie eine erste NGINX-Applikation.

```
kubectl create namespace k8s-lab
kubectl config set-context --current --namespace=k8s-lab
kubectl create deployment web --image=nginx:alpine --replicas=3
```

a) Erstellen Sie den Namespace und setzen Sie ihn als aktuellen Namespace.

```
kubectl create namespace k8s-lab
kubectl config set-context --current --namespace=k8s-lab

kubectl config view --minify | Select-String namespace
```

b) Starten Sie das Deployment «web» mit drei Replicas. Welche Objekte entstehen?

```
kubectl create deployment web --image=nginx:alpine --replicas=3
```

Welche Objekte entstehen dabei? Kubernetes legt eine ganze Objekt-Hierarchie an:

1. **Deployment** (web) — die oberste Ebene, definiert den gewünschten Zustand (Image, Replica-Anzahl, Update-Strategie)
2. **ReplicaSet** (web-xxxxxxxxxx) — wird vom Deployment automatisch erzeugt, kümmert sich darum, dass immer genau 3 Pods laufen
3. **Pods** (web-xxxxxxxxxx-xxxxx) — 3 Stück, jeweils vom ReplicaSet erstellt, jeder mit einem nginx:alpine-Container drin

Zum Anschauen der Hierarchie:

powershell

```
kubectl get deployment,rs,pods
```

c) Prüfen Sie Pods und Deployment. Auf welchen Nodes laufen die Pods?

Tipp: kubectl get pods -o wide / kubectl get deployment

```
kubectl get deployment
kubectl get pods -o wide
```

d) Beschreiben Sie den Weg vom Image «nginx:alpine» bis zu laufenden Pods im Cluster.

Der Ablauf, den du beschreiben sollst, läuft ungefähr so:

1. **API-Request:** kubectl create deployment schickt einen Request an den **kube-apiserver**, der das Deployment-Objekt in **etcd** (bzw. bei k3s: SQLite/Kine als Storage-Backend) persistiert.
2. **Deployment-Controller:** Der **Deployment-Controller** (Teil des kube-controller-manager) erkennt das neue Deployment-Objekt und erstellt daraus ein **ReplicaSet** mit replicas: 3.
3. **ReplicaSet-Controller:** Der **ReplicaSet-Controller** sieht, dass 0 von 3 gewünschten Pods existieren, und erstellt 3 **Pod**-Objekte (noch ohne zugewiesenen Node — Status Pending).

4. Scheduler: Der **kube-scheduler** beobachtet die neuen, noch nicht zugewiesenen Pods und entscheidet anhand von Ressourcen-Verfügbarkeit (CPU/RAM), Node-Selektoren, Affinities etc., auf welchem Node jeder Pod laufen soll. Er schreibt die Node-Zuweisung ins Pod-Objekt (spec.nodeName).

5. Kubelet: Auf dem zugewiesenen Node erkennt das **kubelet** (der Node-Agent) den neu zugewiesenen Pod und beginnt mit dem Start.

6. Container-Runtime: Das kubelet delegiert an die Container-Runtime (bei k3s: **containerd**). Diese:

- lädt das Image **nginx:alpine** vom Docker Hub herunter (Image-Pull), falls es lokal noch nicht vorhanden ist
- erstellt einen Container aus diesem Image
- startet den Container in einem isolierten Namespace (Netzwerk, PID, etc.) via **runc**

7. Netzwerk: Das **CNI-Plugin** (bei k3s: Flannel) weist dem Pod eine Cluster-interne IP-Adresse aus dem Pod-CIDR zu (bei dir z.B. 10.42.0.0/16, das siehst du auch in deinen Server-Logs von vorhin).

8. Status-Reporting: Das kubelet meldet den Pod-Status laufend zurück an den API-Server (via Kubelet → API-Server, nicht direkt in etcd), sodass kubectl get pods den aktuellen Zustand (Pending → ContainerCreating → Running) zeigt.

Kurzfassung für die Antwort in der Aufgabe:

Deployment → ReplicaSet → Pod-Objekte (via Controller-Pattern) → Scheduler weist Node zu → Kubelet auf dem Node zieht das Image via containerd, startet den Container via runc, CNI-Plugin (Flannel) vergibt Pod-IP → Pod wird Running.

Aufgabe 4)

Labels, Selectors und Service

Verbinden Sie das Deployment über einen Service und machen Sie die App lokal erreichbar.

a) Zeigen Sie die Labels der Pods an und filtern Sie alle Pods mit «app=web».

Tipp: `kubectl get pods --show-labels` / `kubectl get pods -l app=web`

```
kubectl get pods --show-labels
```

Das zeigt dir eine zusätzliche Spalte LABELS, in der bei den web-Pods etwas wie `app=web,pod-template-hash=xxxxxxxxxx` steht. Das Label `app=web` wird automatisch vom Deployment vergeben (aus dem Namen `web` abgeleitet), `pod-template-hash` kommt vom ReplicaSet.

Gezielt filtern:

powershell

```
kubectl get pods -l app=web
```

Das nutzt einen **Label Selector** — nur Pods mit exakt diesem Label werden angezeigt. Das ist genau der Mechanismus, den später auch der Service verwendet, um "seine" Pods zu finden.

b) Erstellen Sie einen Service für das Deployment «web».

Tipp: `kubectl expose deployment web --port=80 --target-port=80`

```
kubectl expose deployment web --port=80 --target-port=80
```

Das erstellt einen Service vom Typ ClusterIP (Standard). Kubernetes übernimmt dabei automatisch den Label-Selector `app=web` vom Deployment — der Service "weiss" also selbst, welche Pods zu ihm gehören, ohne dass du das manuell angeben musst.

Prüfen:

powershell

```
kubectl get svc web -o yaml
```

Schau dir dort den Abschnitt `spec.selector` an — dort steht `app: web`.

c) Prüfen Sie Service und Endpoints. Welche Pods werden durch den Service erreicht?

Tipp: `kubectl get svc web` / `kubectl get endpoints web`

```
kubectl get svc web
```

```
kubectl get endpoints web
```

Was passiert dahinter:

- Der **Service** bekommt eine stabile **ClusterIP** (z.B. 10.43.x.x) und lauscht auf Port 80.
- Der **EndpointSlice-Controller** (bzw. der ältere Endpoints-Controller) durchsucht laufend alle Pods im Namespace nach dem Label `app=web` und trägt deren Pod-IPs (aus dem 10.42.0.0/16-Netz) mit Port 80 in die endpoints-Liste ein.

`kubectl get endpoints web` zeigt dir also die **3 Pod-IPs**, die der Service aktuell als Ziel für Traffic-Weiterleitung nutzt — genau die 3 Pods, die `app=web` als Label tragen. Fällt ein Pod aus oder wird neu erstellt, aktualisiert sich diese Liste automatisch.

Zum Gegenchecken, dass es wirklich dieselben Pods sind:

powershell

```
kubectl get pods -l app=web -o wide
```

Die IPs dort sollten mit denen in `kubectl get endpoints web` übereinstimmen.

PS C:\WINDOWS\system32> kubectl get svc web

```
>> kubectl get endpoints web
NAME TYPE      CLUSTER-IP  EXTERNAL-IP  PORT(S)  AGE
web   ClusterIP    10.43.189.83 <none>       80/TCP   37s
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME ENDPOINTS                                AGE
web   10.42.0.6:80,10.42.1.6:80,10.42.1.7:80  38s
PS C:\WINDOWS\system32> kubectl get pods -l app=web -o wide
NAME                                READY  STATUS   RESTARTS  AGE  IP           NODE
web-5d4f7c4d94-9gtz4                1/1    Running  0          24m  10.42.0.6    k3d-teko-k8s-server-0 <none>
web-5d4f7c4d94-rn9s9                1/1    Running  0          24m  10.42.1.7    k3d-teko-k8s-agent-0  <none>
web-5d4f7c4d94-szv7z                1/1    Running  0          24m  10.42.1.6    k3d-teko-k8s-agent-0  <none>
```

d) Machen Sie die App mit Port-Forward lokal erreichbar und öffnen Sie sie im Browser.

Tipp: kubectl port-forward service/web 8080:80

```
kubectl port-forward service/web 8080:80
Das Terminal bleibt jetzt blockiert (läuft im Vordergrund) und leitet lokal localhost:8080 auf Port 80
des Service weiter.
Im Browser öffnen:
http://localhost:8080
Du solltest die Standard-NGINX-Welcome-Page sehen ("Welcome to nginx!").
Zum Beenden des Port-Forwards: Strg+C im Terminal.
Kurz zur Erklärung für den Bericht: Port-Forward baut einen direkten Tunnel vom lokalen
Rechner über den API-Server zum Pod auf (nicht über den Service selbst, auch wenn du
service/web angibst — kubectl löst intern einen der Endpoint-Pods auf und tunnelt zu diesem). Das
ist praktisch zum Testen, aber kein produktionsreifer Zugriffsweg (dafür wären LoadBalancer,
NodePort oder Ingress gedacht).
```

Aufgabe 5)

Skalierung und Selbstheilung

Verändern Sie die Anzahl Pods und beobachten Sie, wie Kubernetes reagiert.

a) Skalieren Sie das Deployment auf fünf Replicas. Was ändert sich?

Tipp: `kubectl scale deployment web --replicas=5 / kubectl get pods -o wide`

```
powershell
kubectl scale deployment web --replicas=5
kubectl get pods -o wide
```

Was ändert sich:

- Das Deployment-Objekt wird aktualisiert: spec.replicas von 3 auf 5.
- Der **ReplicaSet-Controller** bemerkt die Differenz (Ist-Zustand 3, Soll-Zustand 5) und erstellt **2 zusätzliche Pods**.
- Der **Scheduler** verteilt diese 2 neuen Pods wieder auf die verfügbaren Nodes (server-0 und agent-0).
- Die bereits laufenden 3 Pods bleiben unverändert — es wird nichts neu gestartet, nur ergänzt.

Zur Kontrolle:

```
powershell
kubectl get deployment web
Zeigt jetzt READY 5/5.
```

b) Löschen Sie einen Pod aus dem Deployment. Was passiert danach?

Tipp: `kubectl delete pod <pod-name> / kubectl get pods -w`

```
kubectl get pods -l app=web
Dann einen löschen und live beobachten:
powershell
kubectl delete pod <pod-name>
kubectl get pods -w
(-w = watch, läuft live mit, Strg+C zum Beenden)
```

Was passiert:

- Der gelöschte Pod verschwindet (Terminating → weg).
- Der **ReplicaSet-Controller** bemerkt sofort, dass nur noch 4 von 5 gewünschten Pods existieren, und erstellt **automatisch einen neuen Pod** als Ersatz.
- Das ist die **Selbstheilung** (Self-Healing) von Kubernetes: nicht der einzelne Pod wird "repariert", sondern das ReplicaSet stellt einfach den Soll-Zustand wieder her, indem es einen frischen Pod erzeugt. Der neue Pod hat einen neuen Namen/neue IP — er ist nicht identisch mit dem gelöschten, sondern ein komplett neues Objekt.

Wichtig für den Bericht: Das funktioniert, **weil** die Pods über ein Deployment/ReplicaSet verwaltet werden. Ein "nackter" Pod (ohne Controller darüber) würde bei delete einfach verschwinden und nicht neu erstellt werden.

c) Nutzen Sie Describe, Logs und Events zur Diagnose. Welche Informationen sind jeweils hilfreich?

Tipp: `kubectl describe deployment web / kubectl logs deployment/web / kubectl get events --sort-by=.lastTimestamp`

powershell

kubectl describe deployment web

Hilfreich für: Gesamtzustand des Deployments — Soll-/Ist-Replicas, Update-Strategie (RollingUpdate), Selector, und ganz unten ein **Events**-Abschnitt speziell zu diesem Deployment (z.B. "Scaled up replica set web-xxx to 5"). Gut, um zu sehen, was Kubernetes auf oberster Ebene entschieden hat.

powershell

kubectl describe pod <pod-name>

Hilfreich für: Detailzustand eines einzelnen Pods — welcher Node, welche IP, Container-Status (Waiting/Running/Terminated mit Grund), Ressourcen-Requests/Limits, und ebenfalls ein Events-Abschnitt (z.B. "Pulled", "Created", "Started", oder bei Problemen "FailedScheduling", "ImagePullBackOff"). Das ist der wichtigste Befehl, wenn ein Pod nicht hochkommt.

powershell

kubectl logs deployment/web

Hilfreich für: Die eigentliche **Anwendungsausgabe** innerhalb des Containers — bei NGINX z.B. Access-Logs (eingehende HTTP-Requests) oder Fehlermeldungen aus dem NGINX-Prozess selbst. Zeigt standardmässig nur einen der Pods; für alle:

powershell

kubectl logs -l app=web --all-containers --prefix

powershell

kubectl get events --sort-by=.lastTimestamp

Hilfreich für: Chronologischer, **cluster-/namespace-weiter Aktivitätsverlauf** — alle Scheduling-, Pull-, Start-, Stop- und Fehlerereignisse in zeitlicher Reihenfolge, über alle Objekte hinweg. Ideal, um nachträglich zu rekonstruieren, was in welcher Reihenfolge passiert ist (z.B. genau nachzuvollziehen, wie schnell der Ersatz-Pod nach dem delete erstellt wurde).

Aufgabe 6)

Aufräumen und Reflexion

Räumen Sie die Umgebung sauber auf und halten Sie die wichtigsten Erkenntnisse fest.

a) Löschen Sie Namespace oder Cluster. Welche Variante ist gründlicher?

Tipp: `kubectl delete namespace k8s-lab` / `k3d cluster delete teko-k8s`

Namespace löschen:

powershell

`kubectl delete namespace k8s-lab`

Entfernt alle Objekte **innerhalb** dieses Namespace (Deployment, ReplicaSet, Pods, Service, Endpoints). Der k3s-Cluster selbst, alle System-Namespace (kube-system, kube-public etc.) sowie die zugrundeliegende Docker-Infrastruktur (Container, Netzwerk, Volumes) bleiben unangetastet. Gut geeignet, wenn man nur eine Übung/ein Lab zurücksetzen will, den Cluster selbst aber weiter nutzen möchte.

Cluster löschen:

powershell

`k3d cluster delete teko-k8s`

Das ist die **gründlichere** Variante: k3d stoppt und entfernt sämtliche Docker-Container (server, agent, serverlb, tools), das Docker-Netzwerk und die zugehörigen Volumes. Es bleibt buchstäblich nichts vom Cluster übrig — auch keine Config-Dateien, Zertifikate oder System-Namespace. Danach ist der Rechner wieder im Zustand vor `k3d cluster create`.

Kurz für den Bericht: Namespace-Löschung ist ein "logisches" Aufräumen auf Kubernetes-Ebene (innerhalb eines laufenden Clusters), Cluster-Löschung ist ein "physisches" Aufräumen auf Infrastruktur-Ebene (Docker-Container/Netzwerk/Volumes verschwinden komplett). Für ein sauberes Ende der Übung ist `k3d cluster delete` die vollständigere Variante.

Kontrolle danach:

powershell

`docker ps -a`

Sollte keine `k3d-teko-k8s-*`-Container mehr zeigen.

b) Nennen Sie drei Dinge, die bei k3d gleich sind wie bei einem echten Kubernetes-Cluster, und zwei Dinge, die anders sind.

Drei Dinge, die gleich sind:

1. **Die Kubernetes-API und -Objekte sind identisch.** Deployments, ReplicaSets, Pods, Services, Namespaces verhalten sich konzeptionell exakt gleich wie auf einem Cloud-Cluster (AKS, EKS, GKE) — `kubectl` merkt keinen Unterschied, dieselben Manifeste/YAML-Dateien funktionieren 1:1.
2. **Controller-Pattern und Selbstheilung funktionieren gleich.** Scheduler, ReplicaSet-Controller, Self-Healing (wie in Aufgabe 5 beobachtet) laufen genau nach denselben Mechanismen wie in Produktion — k3s ist eine vollwertige, CNCF-zertifizierte Kubernetes-Distribution, kein Simulator.
3. **Netzwerk-Grundkonzepte** (Pod-CIDR, Service-CIDR, CNI via Flannel, DNS über CoreDNS) funktionieren nach demselben Modell wie bei einem "echten" Multi-Node-Cluster.

Zwei Dinge, die anders sind:

1. **Nodes sind Docker-Container, keine echten (virtuellen) Maschinen.** Bei einem Cloud-Cluster ist jeder Node eine eigene VM mit eigenem Betriebssystem-Kernel. Bei k3d teilen sich alle Nodes den WSL2/Linux-Kernel des Host-Systems — das ist der Grund, warum

unser cgroup-v1/v2-Problem überhaupt auftreten konnte (bei echten VM-Nodes hätte jeder Node unabhängig sein eigenes Kernel-Setup).

2. **Kein echter LoadBalancer/Cloud-Provider-Integration.** In der Cloud würde ein Service vom Typ LoadBalancer automatisch eine externe IP von AWS/Azure/GCP bekommen. Bei k3d übernimmt der serverlb-Container (nginx-basiert) diese Rolle nur simuliert lokal auf dem Docker-Host — es gibt keine echte Cloud-Provider-Integration, keine Persistent-Volume-Anbindung an echten Cloud-Storage, und Skalierung ist durch die Ressourcen des lokalen Rechners begrenzt statt durch einen Cloud-Node-Pool.

c) **Notieren Sie Ihre persönliche Kubernetes-Checkliste mit mindestens sechs kubectl- oder k3d-Befehlen.**

Cluster-Lifecycle

powershell

k3d cluster create <name> --agents 2 --api-port 6550

k3d cluster list

k3d cluster delete <name>

Kontext & Namespace

powershell

kubectl config get-contexts

kubectl config set-context --current --namespace=<ns>

Grundzustand prüfen

powershell

kubectl get nodes -o wide

kubectl get pods -A

kubectl get all

Diagnose bei Problemen

powershell

kubectl describe pod <pod-name>

kubectl logs <pod-name> --previous

kubectl get events --sort-by=.lastTimestamp

Skalierung & Deployment-Management

powershell

kubectl scale deployment <name> --replicas=<n>

kubectl rollout status deployment/<name>

kubectl rollout restart deployment/<name>

Netzwerk-Zugriff

powershell

kubectl port-forward service/<name> <local-port>:<port>

Bonus — praktischer Tipp aus heute: Bei k3d immer direkt nach cluster create mit docker ps prüfen, ob wirklich **alle** Container (inkl. serverlb) laufen, bevor man weitermacht — spart viel Debugging-Zeit.